Assignment 3 - Overview

Results, layout, how to run it

14 July 2026

Contents

1 Assignment 3 — Population-Based Search and Decision Making

CSEDU AI Lab. This assignment has **two parts**, and we chose two separate problems rather than bending one problem to fit both. They share a setting: a University of Dhaka residential hall.

	Part A	Part B
Unit	Population-based / swarm	Decision making
Problem	Where do you put 3 Wi-Fi access points on a hall floor?	When do you run the water pump, given Dhaka load-shedding?
Method	Particle Swarm Optimization, from scratch	Value Iteration + Q-learning, from scratch
Code	`src/pso_wifi_placement.py`	`src/rl_water_tank.py`, `src/rl_experiments.py`

The final report is `report/main.pdf` (18 pages, built from the instructor's template). It also covers Assignments 1 and 2 from `../1` and `../2`.

1.1 The two headline results

Part A — the collective is doing the work, not the compute. Everything below runs at an identical 3,030 fitness-evaluation budget:

	mean fitness
1 particle, all 3,030 evaluations to itself	76.78 ± 6.66
30 particles that never communicate (`c2 = 0`)	87.30 ± 1.15
(Random Search, for reference)	*87.43 ± 0.43*
30 particles sharing one `gbest`	**89.91 ± 0.26**

Thirty independent searchers that do not talk are worth no more than throwing 3,030 darts. Restore a single shared number and the same thirty agents, at the same cost, gain 2.47 points (Wilcoxon p = 3.05e-05) and connect every room in the hall instead of stranding one.

But *more* communication is not better, which is a result we took from Kennedy & Mendes (2002) and then had to be corrected by:

topology (same 30 particles, same budget)	mean	std	stagnates
fully connected (`gbest`)	89.83	0.317	iter **84**
ring, k=1	89.96	**0.129**	iter **98**

Over 30 paired seeds the mean difference is **not significant** (Wilcoxon $p = 0.92$) but the variance is — the ring is **6× steadier** (F-test p = 6e-06) and is still improving when the fully-connected swarm has already quit. The ring does not find a *better* answer; it finds a *consistent* one. That is exactly what the paper predicts, and our first draft got it wrong by claiming ”better”.

Part B — we set out to prove something and the data said no. The plan was to show that Q-learning beats a planner working from Dhaka's optimistic published load-shedding schedule. It does not. Ranked by regret against the exact optimum:

	what it knows	regret
Value Iteration, correct model	the exact P	0% (0.23 s)
Value Iteration, **mildly** wrong model	P off by 1.25×	0.1%
Certainty-equivalence VI	$\hat{P}$ estimated from Q-learning's own 720k samples	**1.4% ± 0.2**
Value Iteration, **badly** wrong model	P off by 4×	2.7%
Tuned caretaker threshold rule	nothing; two tuned numbers	14.5%
Q-learning	720,000 samples, no model	15.1% ± 2.4
Value Iteration, believes the grid never fails	no outage model at all	21.4%
Myopic greedy (γ=0)	the model, but no future	120.6%

Read that table twice. **A roughly-right model is worth more than 82 simulated years of experience** — a planner who is merely 25% off still beats the model we *learned* from 720,000 samples. Even a fourfold error beats Q-learning outright. And our Q-learner also loses to a two-parameter threshold rule that does no learning at all.

So the real argument for model-free control is not ”your model might be wrong” — it can be quite wrong and still win. It is that sometimes you cannot write a model down at all. Ours has 1,584 states and we could, which makes this an honest advertisement for Value Iteration and a poor one for Q-learning. Knowing *why* is the point.

(An earlier draft claimed the learned model beat every mis-specified prior. The table we printed directly underneath it said otherwise. The corrected claim is the more interesting one, and the script now computes its conclusion instead of asserting it.)

1.2 Running it

```
pip install -r requirements.txt

./run.sh swarm          # Part A: PSO + Random/Grid baselines + the collective ablation
./run.sh rl             # Part B: Value Iteration vs Q-learning     (~4 min)
./run.sh rl --quick     # Part B, fewer seeds and episodes          (~40 s)
./run.sh test           # 42 unit tests
./run.sh report         # compile report/main.pdf  (the submission)
./run.sh pdfs           # convert every doc in docs/ to PDF -> pdf/
```

Figures land in `results/plots/`, tables in `results/tables/`.

Only `numpy`, `scipy`, `matplotlib` and `pandas`. No DEAP, no PySwarm, no Gym, no stable-baselines. Every algorithm is written from scratch, because the point of a lab is to be able to defend every line.

1.3 Layout

```
3/
├── report/
│   ├── main.tex               # the report, in the instructor's template
│   ├── main.pdf               # 18 pages, covers Assignments 1, 2 and 3
│   ├── references.bib         # all citations verified against Crossref
│   └── figures/
├── src/
│   ├── pso_wifi_placement.py  # Part A: PSO, baselines, collective ablation
│   ├── rl_water_tank.py      # Part B: the MDP, VI, Q-learning, certainty equivalence
│   └── rl_experiments.py      # Part B: experiments, tables, figures
├── tests/                    # 42 tests, including the model-vs-simulator parity gate
├── statement.md               # formal problem statements for both parts
├── docs/
│   ├── START_HERE.md          # <-- read this first if you are new to the project
│   ├── MATH_EXPLAINED.md    # <-- every equation in statement.md, decoded with real numbers
│       ├──  PART1_foundations.md      #  swarm  theory:  NFL,  explore/exploit,  8-
algorithm comparison
│   ├── PART2_applications.md  # 12 cited real-world applications
│   ├── PART3_viva.md          # viva prep, both parts
│   ├── PART4_problem_design.md# the problems we considered and rejected, both parts
│   ├── PART5_code_defense.md  # code-to-theory map + "he points at the screen" questions
│   ├── PART6_decision_making.md # MDP theory: Bellman, contraction, Q-learning convergence
│   └── PART7_literature_applied.md # which papers changed which line, and the 2 that corrected us
├── pdf/                  # every doc above as a PDF, + ALL_DOCS.pdf (95 pages, combined)
├── scripts/build_pdfs.sh      # regenerates pdf/  ->  ./run.sh pdfs
└── results/
    ├── plots/
    ├── tables/
    └── rl_full_run.log
```

The PDFs in `pdf/` are the docs, for reading offline or printing. They are **not** the submission — `report/main.pdf` is.

1.4 Two things worth knowing before you read the code

The room jitter in Part A is not cosmetic. Our first instance had rooms on a perfect rectangular grid, and Grid Search *beat* PSO (89.19 vs 88.41). That result was correct and it was our fault: if the demand points are perfectly gridded, grid-aligned AP positions are exactly where the optimum lives, and we had handed the discrete method the answer. Real rooms are not perfectly gridded. We added positional jitter ($\sigma = 1.5$ m) so the optimum sits *off* any lattice, which is both more realistic and the whole reason to reach for a continuous optimiser.

The diesel ration in Part B is not decoration either. In our first version the generator was unlimited, and mis-estimating the outage rate cost the planner almost nothing — it could always buy its way out one hour later. That made the central experiment measure approximately nothing. Making the diesel a finite daily drum introduces irreversibility: burn it early on a bad forecast and you have none left when the real outage lands. Only then does model error actually hurt.

Both of these were failures first. They are in the report because they are the sharpest things we learned.